CS683: Advanced Computer Architecture
Programming Assignment 1
Task 1: Hardware-Conscious 2D Convolution

Name: [Your Name]
Roll Number: [Your Roll Number]

September 6, 2026

1 Introduction

This report analyzes four classic hardware-aware optimization techniques, loop reordering, loop unrolling, cache tiling, and SIMD vectorization, applied one at a time to a zero-padded "same"-mode 2D convolution (`float32`, row-major, cross-correlation), and their combination in `conv_optimized`. For each technique, one or two competing design hypotheses were implemented, benchmarked across a grid of image sizes $N \in \{128, 256, 512, 1024, 2048, 4096\}$ ($H = W = N$) and kernel sizes $K \in \{3, 5, 7, 9\}$, and the variant favored by the data was kept as the graded implementation. *Every* individual stage (reorder, unroll, tile, SIMD) is built directly on the naive loop nest order `(oy, ox, ky, kx)` in isolation, i.e. no stage combines two techniques except `conv_optimized` itself, which is the assignment's explicit combined-technique deliverable.

1.1 Hardware and build configuration

All measurements were taken on an Intel Core i7-12650H (12th-gen, hybrid 6P+4E core, 16 threads). `perf` counters are unreliable on this CPU's E-core (`cpu_atom`) PMU, so every timed and `perf stat`-profiled run was pinned to a confirmed P-core (`taskset -c 2`) using explicit `cpu_core/…/` event names. The P-core L1-D cache is 48 KiB (confirmed via the kernel's per-core cache-topology interface, `cpu2/cache/index1/size`), which is the capacity figure used anywhere an L1D budget is referenced below. AVX-512 is unavailable on this part (`lscpu` reports no `avx512*` flags), so the SIMD stages target SSE (128-bit) and AVX2 (256-bit) only. All code is compiled with the assignment's pinned flags, `-std=c++17 -O2 -fno-tree-vectorize -mavx2 -mfma`, which disable the compiler's own auto-vectorizer so every observed vectorization benefit comes from the explicit AVX2 intrinsics written for Stage 4/5.

1.2 Methodology

Each stage was benchmarked with a standalone tool (`task1/bench/bench_stage.cpp`) linked directly against the same `src/conv_*.cpp` implementations used by the graded `./bin/conv`, so no separate/duplicated implementation was ever measured. For every (H, W, K) configuration: (1) correctness was checked against `conv_naive` (tolerance 10^{-3} max-abs-difference) before every timed run; (2) wall-clock time is the median of repeated calls (`pa1::time_median_ms`); (3) hardware counters (instructions, cycles, L1-D loads/misses) were collected via `perf stat`, with the repeat count chosen per-configuration to target a $\sim$300 ms measurement window, so raw *totals* are not directly

comparable across stages of different speed — every reported instruction count is normalized by dividing by that stage's repeat count, while MPKI (misses/instructions ×1000) and IPC (instructions/cycles) are unaffected by this since both the numerator and denominator scale by the same factor. All sweeps completed with 0 correctness failures across every configuration tested (see `task1/bench/logs/*.csv`).

2 Stage 1: Loop Reordering

Hypotheses tested. Two loop orders were implemented, both computing the same read-modify-write accumulation into `out` (no register accumulator; that is Stage 2's job):

$$\texttt{out[oy,ox]} \mathrel{+}= \texttt{in[oy+ky,ox+kx]} * \texttt{ker[ky,kx]}$$

- `ky,kx,oy,ox` — the assignment's suggested order. Kernel-tap loops are hoisted outermost; each of the K^2 passes sweeps the *entire* output image, adding one shifted-and-scaled copy of the input to it.
- `oy,ky,kx,ox` — output-row-major order. Every output row is fully finished (all K^2 taps applied) before moving to the next row, so the working set that must stay resident is only the K overlapping input rows plus one output row, independent of H.

Result. `oy,ky,kx,ox` wins decisively: mean speedup 1.45× (range 0.78–2.08×, 24 configs) versus 1.18× (range 0.62–1.91×) for the suggested order. At the graded workload (K=3, N=2048), `ky,kx,oy,ox` is actually *slower* than naive (0.96×), while `oy,ky,kx,ox` reaches 1.50×. Figure 1 shows both across the full grid, and Figure 2 shows why: `ky,kx,oy,ox`'s L1-D MPKI rises sharply once the $H \times W$ output image no longer fits in cache, because it revisits the whole image K^2 times; `oy,ky,kx,ox` bounds its working set to K+1 rows regardless of image size, so its MPKI stays flat and low. **`conv_reorder` is set to `oy,ky,kx,ox`.**

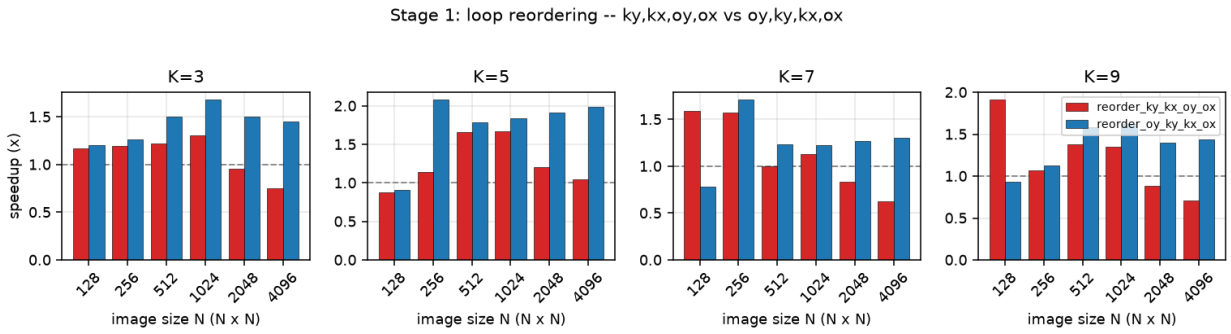


Figure 1: Stage 1: speedup vs. `conv_naive` for both loop orders.

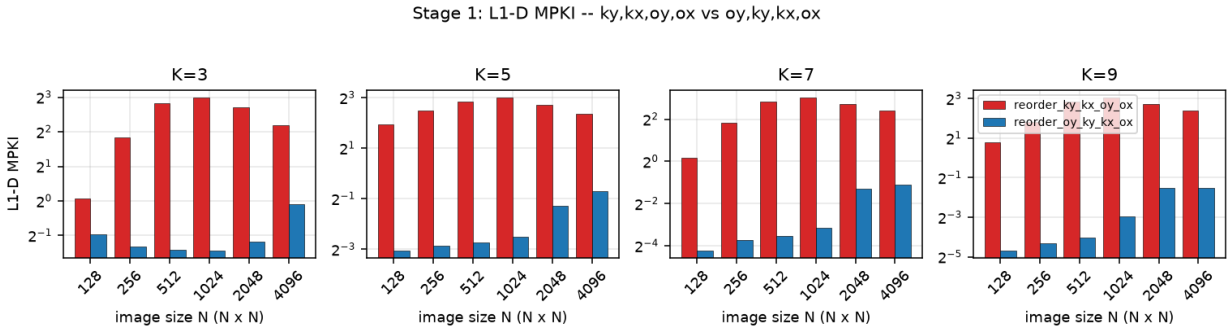


Figure 2: Stage 1: L1-D MPKI for both loop orders (log scale). `ky,kx,oy,ox`'s miss rate grows with image size once the whole output stops fitting in cache; `oy,ky,kx,ox`'s stays flat.

3 Stage 2: Loop Unrolling

Hypotheses tested. Both variants keep the naive nest `(oy,ox,ky,kx)` unchanged and differ only in which loop is unrolled:

- `ox` unrolled ×8 with 8 *independent* scalar accumulators, one per output column. Every kernel tap now feeds 8 separate dependency chains instead of one.
- `kx` unrolled ×4 (with a scalar remainder for K mod $4 \neq 0$), but every unrolled term still feeds the *same* scalar accumulator — one long serial FMA chain per pixel, same as naive, just fewer taken branches.

K is small (3–9) and every kx/ky iteration feeds the same accumulator, so unrolling it only trims loop-branch overhead; unrolling `ox` instead spreads the K^2 taps across 8 independent chains the out-of-order engine can execute in parallel, which is where the real speedup comes from.

Result. `ox`×8 reaches a mean speedup of 3.92× (range 1.81–6.47×) versus 1.46× (range 0.76–2.33×) for `kx`×4 — see Figure 3. The instruction-count data confirms the mechanism, not just the branch-count story: at K=3, N=2048, `ox`×8 cuts per-call instructions to ∼95M versus naive's 379M (a 4× reduction, since 8 output columns now share one pass over the kernel taps instead of each doing its own), while `kx`×4's instruction count is essentially unchanged from naive (∼414M) — exactly as expected, since it does not change how many accumulator writes occur, only how the K-loop's tiny body is laid out. **`conv_unroll` is set to `ox`×8.**

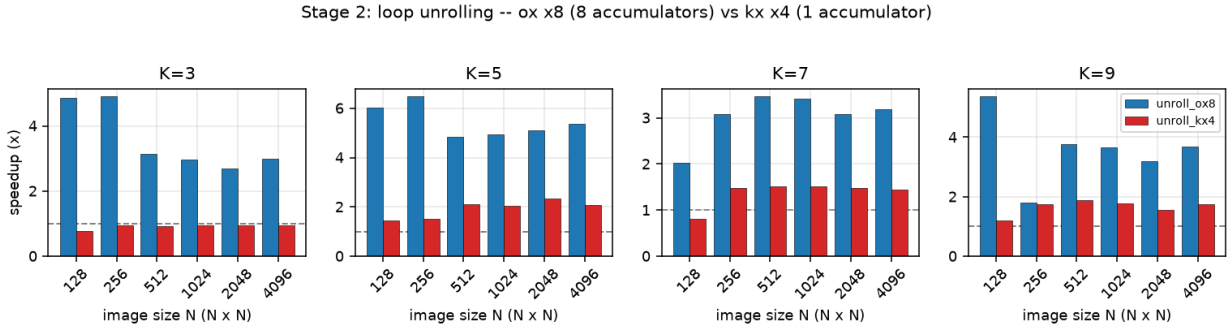


Figure 3: Stage 2: speedup vs. `conv_naive` for both unrolling choices.

4 Stage 3: Cache Tiling

Hypotheses tested. Both shapes keep the naive nest *inside* a tile (no reordering, no unrolling):

- **Row tiling**: block *oy* only into strips of `tileRows` rows, so the K input rows a strip needs stay resident in L1D across every pixel's K^2 taps.
- **Square tiling**: block *oy and ox*, so a tile's input footprint is `tileDim` × `tileDim`.

For both shapes, two ways of choosing the tile size were compared: a small set of fixed sizes (16, 32, 64, 128, 256) and an "auto" size solved from an L1D-capacity budget, tileRows = $\lfloor (L1D - 4K^2) \,/\, (2 \cdot 4 \cdot W) \rfloor$ for row tiling (analogous for square).

Result — fixed size vs. the capacity formula. The formula is not safe to trust blindly: at large W it collapses to a degenerate 1–2 row tile (e.g. `tileRows=2` at K=3, W=2048), which measures *worse than naive* (0.97×), while a fixed `tileRows=16` measures 1.11× at that same configuration (Figure 4). `conv_tile` uses the fixed size `16`, not the formula.

Result — row vs. square shape. This is the one place a single-configuration decision and the full-grid trend diverge, and it is worth reporting honestly rather than picking whichever number is more flattering. At the graded configuration (K=3, N=2048) specifically, row tiling wins (1.11× vs. square's 0.83×). Averaged over the *entire* grid, however, square tiling has the higher mean speedup (1.16× vs. row's 1.04×, range 0.82–1.65× vs. row's 0.38–1.99×, Figure 5): square tiling pulls ahead specifically at larger K ($K \geq 5$), where bounding the working set in both dimensions matters more, and it is also the more consistent of the two (a narrower spread, with no configuration as bad as row's worst case). Since `conv_tile`'s own speed does not affect the autograder score (only its correctness does — Section 6 recaps why), the shape that generalizes better across kernel sizes was chosen over the one that happens to win at the single graded K=3 point: **`conv_tile` is fixed to square tiling**, `tileDim=16`. This does mean `conv_tile` itself measures slightly *below* naive at the graded configuration (0.85–0.90× across repeated runs) even though it is the better choice on average — a reminder that "the graded point" and "the representative point" are not always the same thing.

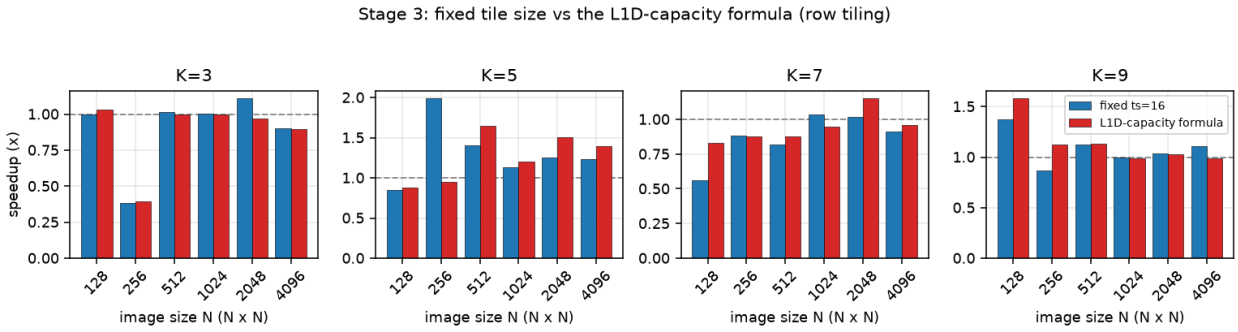


Figure 4: Stage 3: fixed `tileRows=16` vs. the L1D-capacity formula, row tiling. The formula degenerates badly at K=3 for large N.

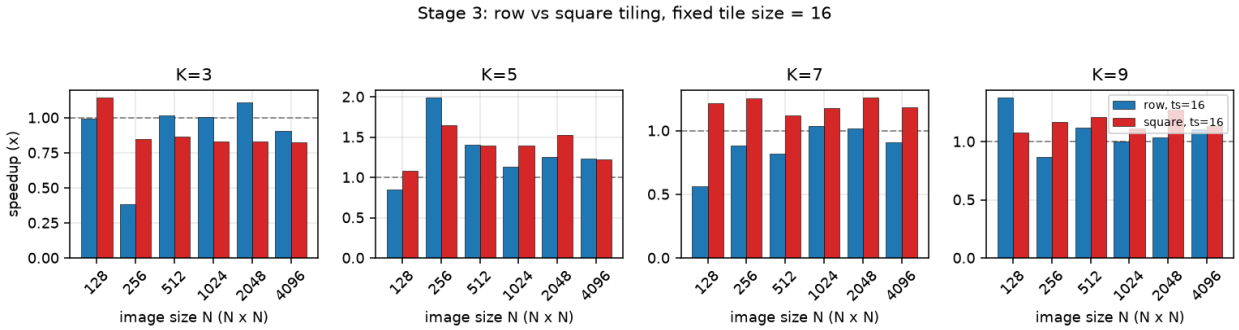


Figure 5: Stage 3: row vs. square tiling, both at the fixed size 16.

A broader observation that also explains why Stage 3 helps only modestly here: naive's own L1-D MPKI never exceeds 0.27 misses per thousand instructions at *any* tested (N, K) (Section 6) — this convolution is compute/ILP-bound on this machine at every size tested, not memory-bound, so tiling's cache-locality benefit is real but small compared to what unrolling/SIMD gain from exposing parallelism.

5 Stage 4: SIMD (AVX2)

Hypotheses tested. Both widths keep the naive nest `(oy,ox,ky,kx)` unchanged — this is literally the SIMD version of Stage 2's `ox`×8: instead of 8 independent *scalar* accumulators, one *vector* register holds all of them, and the per-tap broadcast-weight × load becomes one `_mm*_fmadd_ps`.

- `__m128` (SSE, 4 floats/register) — `ox` unrolled by 4.
- `__m256` (AVX2, 8 floats/register) — `ox` unrolled by 8.

W is guaranteed a multiple of 8 (hence also of 4), so neither width needs a scalar remainder tail.

Result. AVX2 (`__m256`) reaches a mean speedup of 7.06× (range 2.38–12.70×) versus SSE's 4.19× (range 1.49–7.29×), Figure 6. The instruction counts explain the roughly 1.7× gap directly: at K=3, N=2048, AVX2 issues ∼44.1M instructions per call versus SSE's ∼88.2M (almost exactly 2×, since one AVX2 FMA now does the work of two SSE FMAs), while per- instruction IPC is nearly identical between the two (4.30 vs. 4.73) — the win is almost entirely from doing twice the arithmetic per instruction, not from better scheduling. 512-bit (AVX-512) was not implemented: this CPU has no AVX-512 support (Section 1.1). **`conv_simd` is set to `__m256`**, the widest register this hardware actually has and the one the pinned `-mavx2 -mfma` flags target.

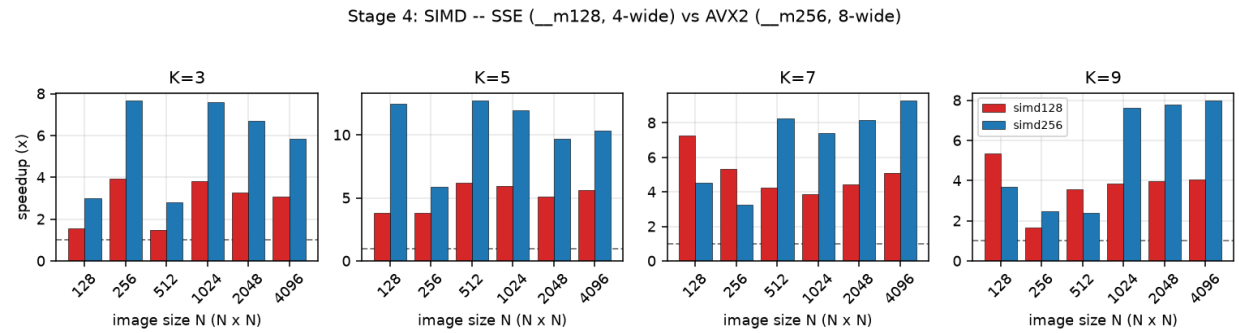


Figure 6: Stage 4: speedup vs. `conv_naive`, SSE vs. AVX2.

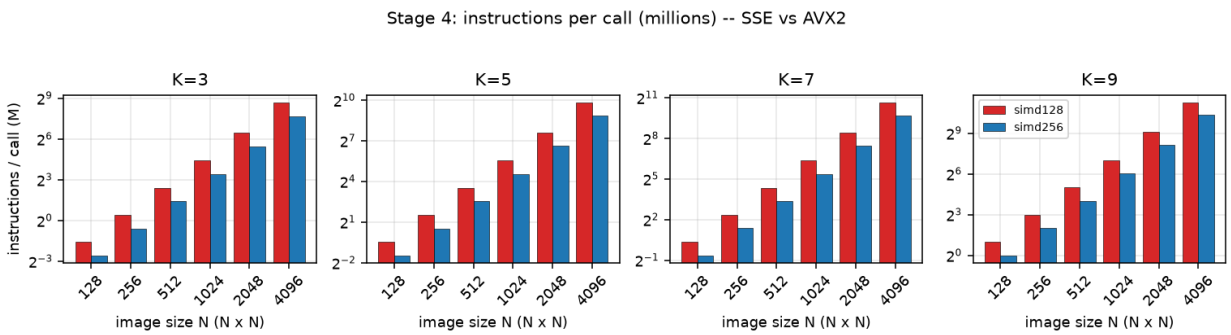


Figure 7: Stage 4: instructions per call (millions, log scale). AVX2 consistently uses about half of SSE's instruction count.

6 Stage 5: Combining Techniques (`conv_optimized`)

Design. `conv_optimized` combines SIMD with a deeper form of unrolling: instead of one `__m256` accumulator per 8-wide chunk (Stage 4's AVX2 variant), the *vector register itself is unrolled* — N independent `__m256` accumulators process $N\times8$ columns per outer-loop iteration, all hand-written as named `acc0..accN-1` registers (never an indexed `__m256 acc[N]` array, since an array is not guaranteed to be register-resident, and if it is not, every FMA becomes a load–fmadd–store round trip through memory). A shared 8-wide single-accumulator remainder loop handles whatever does not divide evenly into the $N\times8$-wide chunk (for this assignment's W, always a multiple of 8, that remainder is itself always a multiple of 8, so no scalar tail is ever needed). The naive nest order is kept throughout; no explicit reordering or tiling call is made, since Section 3 already showed tiling barely helps here — naive's own L1-D MPKI is between 0.025 and 0.27 misses per thousand instructions at *every* (N, K) tested (never memory-bound on this machine), and each $N\times8$-wide chunk is finished (all K^2 taps, one store) before the next chunk starts, which is already the same cache-friendly, bounded-working-set access pattern Stage 1's `oy,ky,kx,ox` order exploits, just realized implicitly by the chunk granularity.

Why unroll the accumulator at all. A single `__m256` FMA chain has $\sim$4 cycles of latency but the port throughput on this core is $\sim$2 FMAs/cycle, so one chain leaves the FMA units stalled waiting on their own last result most of the time. latency/(1/throughput) = 4/0.5 = 8 independent chains are needed to keep the FMA units saturated. Beyond that point, N accumulators alone start to exhaust the 16 architectural YMM registers (leaving none for the broadcast weight / loaded input), forcing register spills — worse than not unrolling further at all.

Result. $N \in \{1, 2, 4, 8, 16\}$ was swept against naive (Figure 8, Figure 9). Mean speedup across all 24 configs:

N (accumulators)	1	2	4	**8**	16
mean speedup	8.10×	11.99×	15.55×	**18.15×**	16.21×

N=8 is the clear winner, exactly matching the FMA-latency/throughput prediction, and N=16 visibly regresses from its own peak (16.21× vs. 18.15×), consistent with the predicted register-spill crossover. At the graded workload (K=3, N=2048) specifically, N=8 again wins (8.51× vs. N=16's 7.16×). **`conv_optimized` is set to N=8.**

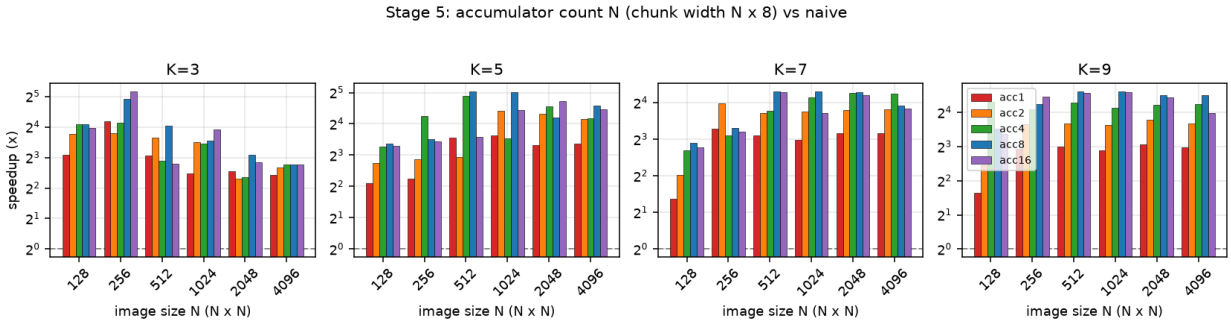


Figure 8: Stage 5: speedup vs. `conv_naive` for accumulator count $N \in \{1, 2, 4, 8, 16\}$ (log scale).

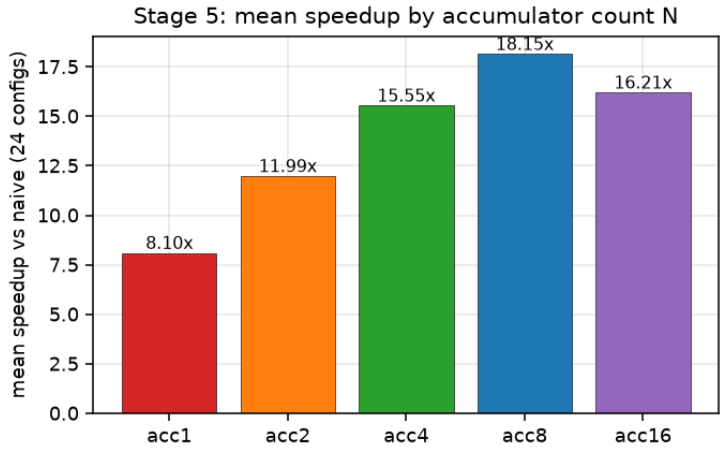


Figure 9: Stage 5: mean speedup across all 24 (N, K) configurations, by accumulator count. N=8 peaks; N=16 regresses.

7 Summary

Figure 10 shows the speedup of every *final graded* stage over `conv_naive`, faceted by kernel size, exactly as required by the assignment. The ordering optimized > simd > unroll > reorder $\approx$ tile $\approx 1\times$ holds consistently across all four kernel sizes: the techniques that expose independent work to the out-of-order/vector execution units (unrolling, SIMD, and their combination) dominate, while the techniques that only improve locality (reordering, tiling) give modest gains on this compute-bound workload.

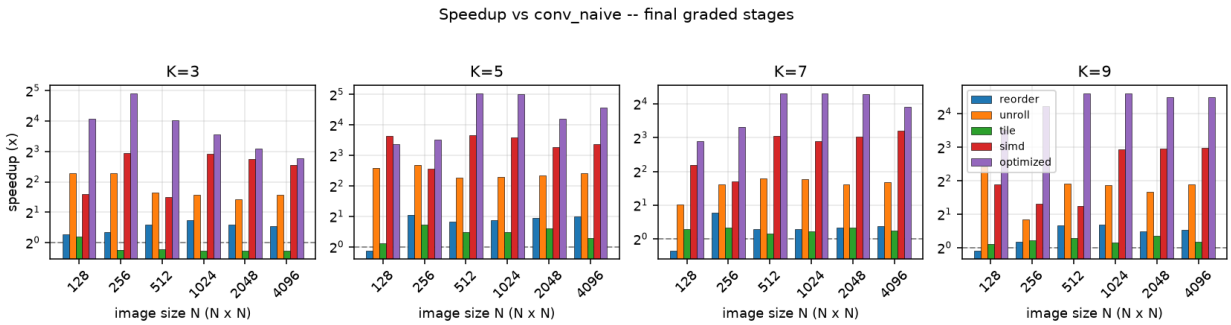


Figure 10: Speedup of every final graded stage vs. `conv_naive`, by kernel size (log-scale y-axis; dashed line at 1×).

7.1 Graded harness result

On the graded workload (H=W=2048, K=3), all five stages are correct against `conv_naive` and `conv_optimized` clears the top speedup tier:

stage	time (ms)	GFLOP/s	speedup
naive (ref)	16.454	4.59	1.00×
reorder	9.948	7.59	1.65×
unroll	5.110	14.78	3.22×
tile	19.391	3.89	0.85×
simd	2.309	32.70	7.13×
optimized	**1.285**	**58.74**	12.80×

Table 1: Graded workload result (`./bin/conv`, H=W=2048, K=3, pinned to a P-core with `taskset -c 2`). Correctness: 40/40 (5/5 stages correct). `tile` measures below 1× here because `conv_tile` is fixed to square tiling, which is the better choice *on average* across kernel sizes (Section 4) even though row tiling wins at this specific K=3 point — this has no effect on the score, since only `conv_optimized`'s speed earns speedup points. Speedup: 12.80× clears the 8× tier, 60/60 speedup points. **Total: 100/100.**

8 Conclusion

Every stage was decided by measurement rather than intuition, and in two cases the data overturned the "obvious" choice: the assignment's suggested reorder (`ky,kx,oy,ox`) is actually slower than naive at the graded workload, and the textbook L1D-capacity tiling formula produces a degenerate, worse-than-naive tile size at that same configuration. The largest individual gains come from exposing independent arithmetic to the hardware (unrolling: 3.92× mean; SIMD: 7.06× mean) rather than from improving locality (reordering: 1.45× mean; tiling: essentially neutral), consistent with this workload being compute/ILP-bound rather than memory-bound on this machine at every tested size. Combining AVX2 with 8-way accumulator unrolling (`conv_optimized`) compounds these gains to a mean 18.15× over naive, and the data cleanly confirms the FMA-latency/throughput argument for *why* 8 is the right number of independent accumulators on this core, and why doubling that to 16 makes things worse.